

CSIS 604 - Distributed Systems - Midterm

A Discussion of Controversial Topics in Distributed Systems

Dan Johnson

Midterm

- Midterm
 - 1. Robustness of public peer-to-peer systems
 - * Prompt
 - * Recommendation
 - 2. Edge Computing
 - * Prompt
 - * Recommendation
 - 3. Virtual Machines Versus Container Technology
 - * Prompt
 - * Recommendation
 - 4. Scalability of publish-subscribe systems
 - * Prompt
 - * Recommendation
 - 5. Scalability of blockchain systems
 - * Prompt
 - * Recommendation
 - References

1. Robustness of public peer-to-peer systems

Prompt

The first wave of public peer-to-peer systems has passed and by now most are operating within the boundaries of a single organization. This shift reflects the difficulty in getting public peer-to-peer systems secured against, notably Eclipse and Sybil attacks. Nevertheless, there are still a number of systems that operate in the open, such as the DHT-based system for so-called magnet links (used by, for example, Bittorrent), as well as the Kademlia system.

Understanding the ins and outs of various types of peer-to-peer systems, you are asked to advise a new startup on developing a fully decentralized file-sharing system on whether or not they should base this system on an existing product. One particular concern they have is the robustness against churn, for which they are thinking of using a gossip-based approach which are known to quickly converge to specific overlays, yet are vulnerable to attacks.

Recommendation

Development of a fully decentralized peer-to-peer (P2P) file-sharing system from scratch is a *monumental* engineering effort for an established organization to undertake, let alone a humble startup. For this reason, the startup should base their infrastructure around a churn-resistant and security-hardened framework like the Bamboo DHT.¹ Bamboo was specifically designed to handle extreme session churn rates that are historically comparable to real-world P2P file-sharing networks,² proving its resilience as the underlying substrate for public deployments like OpenDHT.³ Leveraging this structurally optimized routing layer allows the startup to focus development resources on application-layer features while ensuring baseline predictability and hardening the system against routing attacks.

While the startup's inclination to exploit an unstructured, gossip-based approach correctly solves the problem of rapid convergence while mitigating node churn, this approach introduces severe structural vulnerabilities in open or public environments. Due to the gossiping protocol's reliance on probabilistic (non-deterministic) neighbor exchanges to maintain topology, implementations that rely on it inherently lack the formal cryptographic verification constraints required to secure routing state or data stewardship necessary to mitigate malicious node introduction.⁴ Consequently, protocols including the pervasively popular Kadem-

¹Sean Rhea et al., "Handling Churn in a DHT," *Proceedings of the USENIX Annual Technical Conference*, 2004, <https://www.cs.princeton.edu/courses/archive/fall09/cos518/papers/bamboo>.

²Rhea et al., "Handling Churn in a DHT."

³Sean Rhea et al., "OpenDHT: A Public DHT Service and Its Uses," *Proceedings of the 2005 ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '05)* (New York, NY, USA), 2005, 73–84, <https://doi.org/10.1145/1080091.1080102>.

⁴Guido Urdaneta et al., "A Survey of DHT Security Techniques," *ACM Comput. Surv.* (New York, NY, USA) 43, no. 2 (2011), <https://doi.org/10.1145/1883612.1883615>.

lia protocol are “still vulnerable to Sybil and Eclipse attacks as nodes can generate their own identifiers”.⁵ Notably, Bamboo mitigates this to an extent by using a hash of the node’s IP address as the key,⁶ e.g. since nodes are disallowed from choosing their own keys under this protocol, a malicious node can not simply spoof a key when joining.

In order to reconcile these adversarial risks to data consistency within a file-sharing context, the architecture must prioritize the Consistency (C) and Partition Tolerance (P) of the CAP Theorem over unconditional Availability (A).⁷ While geographically distributed systems routinely sacrifice consistency to maximize throughput, a public file-sharing system can not afford to return unverified or corrupted data chunks during period of network instability. Using a structured framework like Bamboo allows the implementation the leverage of deterministic routing paths to safely achieve eventual consistency.⁸ By utilizing Bamboo’s fixed, periodic failure recovery mechanisms along with an explicit cryptographic verification layer - such as validating each data chunk against its immutable hash much like the venerable Bittorrent⁹ or requiring cryptographic signatures on **puts** - the platform can preserve strict data integrity even across highly partitioned networks.

⁵Urdaneta et al., “A Survey of DHT Security Techniques,” p 47.

⁶Urdaneta et al., “A Survey of DHT Security Techniques,” p 48.

⁷Seth Gilbert and Nancy Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News* 33, no. 2 (2002): 51–59, <https://doi.org/10.1145/564585.564601>.

⁸Urdaneta et al., “A Survey of DHT Security Techniques,” p 48.

⁹Arvid Norberg, *Introduction to BitTorrent*, Distributed systems C, TDBC85, Umeå University, 2006, <https://www.libtorrent.org/bittorrent.pdf>, p 9.

2. Edge Computing

Prompt

One of your fellow students is considering setting up a startup for developing an edge- computing platform that can be easily connected to a range of cloud-service providers. One of the key issues your colleague sees is not only provisioning the basic infrastructure, but also a complete set of tools that will automatically manage resource allocation for the applications to be allocated to the edge. Many of those applications either currently run on edge devices or in the cloud.

Your expert team is aware of the hype around edge-computing systems and notably the discussions on the apparent advantages concerning security, performance, etc. You also notice that your colleague student may not be aware of the ins and outs and you decide to give a substantiated advice on whether or not she should carry on with her ideas, and if so, what the main issues are that she should consider to make the startup attractive from an end-user’s perspective.

Recommendation

While edge computing architectures offer some compelling advantages for certain applications such as low latency, location awareness, and reduced WAN bandwidth¹⁰, there are also extensive trade-offs which make these systems a poor fit for other applications. As such, the architectural complexity of building a cross-provider orchestration platform which automatically manages applications to the edge ranges from challenging at best to prohibitive at worst. The core impediment to such a business model is the reality of hardware, software, and API heterogeneity which is the “main challenge in the successful deployment of Edge computing”.¹¹ A startup attempting to deploy a single, universal control plane would likely drown in technical debt due to interoperability limitations.

To mitigate the extreme interoperability tax, the startup should pivot the platform’s core architecture towards a fog computing paradigm.¹² Whereas pure edge models struggle against a fragmented ocean of heterogeneous client hardware devices, fog computing seamlessly extends cloud services down to uniform network infrastructure like “edge routers, switches, gateways, and access points, PCs, smartphones, set-top boxes, etc.”¹³ While heterogeneity is *still a factor* in fog computing¹⁴, the relative homogeneity of the underlying computing hardware (such as x86 vs ARM architecture), network communication protocols, etc.

¹⁰Wazir Zada Khan et al., “Edge Computing: A Survey,” *Future Generation Computer Systems* 97 (2019): 219–35, <https://doi.org/https://doi.org/10.1016/j.future.2019.02.050>, pp 221–222.

¹¹Khan et al., “Edge Computing,” p 222.

¹²Carla Mouradian et al., “A Comprehensive Survey on Fog Computing: State-of-the-Art and Research Challenges,” *IEEE Communications Surveys & Tutorials* 20, no. 1 (2018): 416–64, <https://doi.org/10.1109/COMST.2017.2771153>.

¹³Mouradian et al., “A Comprehensive Survey on Fog Computing.”

¹⁴Mouradian et al., “A Comprehensive Survey on Fog Computing,” p 426.

is orders of magnitude smaller than in the world of edge IoT devices. Shifting the target orchestration plane to these more-standardized conventional devices eliminates the need to develop multitudes of custom software variations for highly localized, ever-changing devices. Furthermore, by leveraging pre-existing, virtualized network layers to manage resource allocation, the startup preserves deterministic, scalable execution trees while silently handling the complexity of state reconciliation underneath.¹⁵

From the perspective of the end-user, the transition of focus from raw edge hardware provisioning to a managed fog service layer delivers *immediate* operational value. Enterprise consumers prioritize distributed deployments to enforce local data sovereignty, maintain privacy boundaries and guarantee offline autonomy during network failures, not to micromanage heterogeneous physical infrastructure.¹⁶ Exposing a standardized fog layer as a secure, predictable application, the startup shields end-users from the underlying complexity of hardware maintenance and network jitter.¹⁷ Ultimately, treating the platform as a modular, privacy-preserving extension of a customer’s pre-existing cloud architecture creates a highly attractive, low-cost integratin model that directly mitigates enterprise deployment risk.¹⁸

¹⁵Mouradian et al., “A Comprehensive Survey on Fog Computing.”

¹⁶Khan et al., “Edge Computing,” p 229.

¹⁷Khan et al., “Edge Computing,” p 221.

¹⁸Khan et al., “Edge Computing,” p 230.

3. Virtual Machines Versus Container Technology

Prompt

A small data center that operates for a largely regional, yet functionally wide range set of companies, is considering a complete transition to container technology instead of their current use of virtual machines. Their main reason is the suspected gain in performance and scalability (so they say). Their customers rely on a diverse set of applications, requiring the need for supporting office-based operating systems such as Windows, but also a range of different Unix machines. So far, the data center has used Debian distributions as their base operating system.

Your expert team is more than just knowledgeable when it comes to virtualization and is very much aware of the strengths and limitations of containerization. Instead of advising what to do, you offer to enhance the awareness around containerization and virtualization so that the data center operators can make a founded decision on if, where, and how to apply containers. You suspect that in the end, they may need to use both containers and virtual machines, side-by-side. Providing the proper requirements for whatever solution they decide on, is what you will be offering.

Recommendation

The primary distinction between a virtual machine and a container is that a virtual machine *mimics the underlying hardware and operating system of a physical machine on a host machine* whereas a container is fundamentally *an isolated (containerized) process running on the host machine which virtualizes only the operating system*.¹⁹ This distinction is important when choosing which technology to base a particular application or infrastructure on because the containerized processes “do not run their own OS kernels, but instead rely on the underlying kernel for OS services.”²⁰ Due to this, containerization applications are a non-starter if the firm is trying to host a Windows application (which will rely on the Windows kernel) on a Debian or Unix host, at least without an additional layer of virtualization.

As for the perceived performance gains that containerization touts over full hardware virtualization, “containers have a reputation for substantially better performance than virtual machines, however that reputation may not be deserved.”²¹ It appears that these performance gains are dependent on the type of *work* the underlying application may be doing, as containers *do* have a performance edge *when the applications are I/O bound*.^{22,23} This is likely due to the I/O bound applications use of a *system call*, but notably when

¹⁹Prateek Sharma et al., “Containers and Virtual Machines at Scale: A Comparative Study,” *Proceedings of the 17th International Middleware Conference* (New York, NY, USA), Middleware '16, 2016, <https://doi.org/10.1145/2988336.2988337>.

²⁰Sharma et al., “Containers and Virtual Machines at Scale,” p 2.

²¹Allison Randal, “The Ideal Versus the Real: Revisiting the History of Virtual Machines and Containers,” *ACM Computing Surveys* 53, no. 1 (2020): 1–31, <https://doi.org/10.1145/3365199>, p 14.

²²Randal, “The Ideal Versus the Real,” p 14.

²³Sharma et al., “Containers and Virtual Machines at Scale.”

we can minimize these system calls (e.g. for applications that remain in user space), the performance of VMs approaches that of the host machine.²⁴

From a security standpoint, the fact that a container must make several privileged system calls to create its isolated environment in the first place means that containers do have a larger vulnerability surface than virtual machines.²⁵²⁶ It is in the area of security that the firm is likely to incur costs as vulnerabilities inevitably expose the firm to legal and financial risks. To mitigate this multi-tenant security risk while still capitalizing on agile application scaling, the firm should offer a hybrid, side-by-side architecture that runs multiple-process container runtimes nested within hardware-isolated virtual machine silos.²⁷ This nested configuration delivers the robust, multi-OS security boundaries of hypervisors to satisfy both Windows and Unix clients, while concurrently providing low-overhead packaging and rapid cloning of containers within trusted, isolated, and secure boundaries.²⁸

²⁴Martinus Richardus van Steen and Andrew S. Tanenbaum, *Distributed Systems* (Amazon Fulfillment Poland Sp. z o.o, 2017), pp 120-121.

²⁵Randal, “The Ideal Versus the Real,” p 14.

²⁶Sharma et al., “Containers and Virtual Machines at Scale,” p 8.

²⁷Sharma et al., “Containers and Virtual Machines at Scale,” p 10.

²⁸Sharma et al., “Containers and Virtual Machines at Scale,” p 10.

4. Scalability of publish-subscribe systems

Prompt

As with so many hypes, lots of folks have started to ride the wave of publish-subscribe systems without really understanding what is going on. Fortunately, your expert team does understand what is in that wave. You have been hired by a company (called REPS) that wants to set up a nationwide competitor to `funda.nl` for selling and renting a range of real estate. Their unique selling point is that a customer can subscribe to new property that matches their wishes such that within only a few seconds they will receive a notification when that property comes available.

You immediately understand that you're dealing with a content-based publish-subscribe system, but with the current turnover in the housing market, you also understand that scaling may be an issue. Moreover, missing out on a notification may lead to claims by customers.

To this end, you decide to dig further into the matter and identify the tradeoffs REPS needs to consider. You ask yourself whether a simpler topic-based or channel-based pub-sub system can suffice, or perhaps a combination of the two. In any case, you will need to make clear to REPS what they are facing up to. To make matters worse, REPS has decided that it would prefer to also guarantee that matches remain anonymous, allowing them to make use of existing cloud-based services without the need to fully trust those services. If time allows, you will also advise on this additional requirement, knowing that it is not going to help keeping matters simple.

Recommendation

While content-based publish-subscribe systems offer maximum expressiveness by evaluating multiple attribute filters against a wide range of event contents, implementing this paradigm for a nationwide `funda.nl` competitor introduces a severe matching bottleneck. If the subscriber base s and property listings p grow linearly, a naive linear search requires $\Theta(s \cdot p)$ comparisons runtime overhead.²⁹ To put this in perspective, `funda` carries about 4.6 million users³⁰, so if we assume that $s = p$ and REPS is competing for the same pool of users and properties, then there is potential for 16 trillion comparisons for such a system with a deadline of only a few seconds, which will lead to costly claims by customers over missed notifications. Pivoting to topics has severe trade-offs as well, as either the subscriber (client) would need to filter out irrelevant topics leading to an inefficient use of bandwidth, or the topics would need to be split out into a multitude of subtopics recursively which has the unfortunate side-effect of producing redundant events³¹ and can also introduce scaling issues.

To resolve this algorithmic bottleneck, the company should abandon exact, centralized evaluations in fa-

²⁹Patrick Th. Eugster et al., “The Many Faces of Publish/Subscribe,” *ACM Comput. Surv.* (New York, NY, USA) 35, no. 2 (2003): 114–31, <https://doi.org/10.1145/857076.857078>, pp 8-10.

³⁰Funda Engineering, *About Funda*, <https://blog.funda.nl/about/>, 2026.

³¹Eugster et al., “The Many Faces of Publish/Subscribe,” p 10.

vor of a distributed routing topology backed by probabilistic pre-filtering layers.³² By deploying an array of brokers that utilize localized subscription containment posets or Bloom-filter arrays, the system can quickly discard non-matching event spaces before executing expensive downstream predicate checks.³³ Alternatively, transitioning the compute model to a probabilistic routing vector or a two-way recommendation engine allows the platform to sacrifice absolute completeness within a small, quantifiable error bound to drastically minimize real-time matching latency.³⁴ This architectural compromise shifts the system away from fragile, synchronous evaluation bottlenecks and moves it toward a highly parallelizable framework capable of executing wide-area content delivery.

The constraint to preserve subscriber anonymity across untrusted cloud providers adds intense security complexity, introducing severe vulnerabilities to overlay flooding and subscription-leak attacks.³⁵ To satisfy this criteria without full cloud trust, the architecture must implement a secure proxy anonymizer engine that intercepts user queries and cloaks them within an aggregate anonymity set of obfuscated and false subscriptions.³⁶ While this prevents third-party hosts from inferring exact consumer interest, it intentionally inflates the routing table volume and introduces a massive risk of bad actors launching malicious oversubscription attacks to starve system resources. Therefore, guaranteeing true identity and subscription secrecy requires the startup to implement strict role-based access control (RBAC) and out-of-band cryptographic token verification at edge boundaries to keep malicious actors from hijacking or poisoning the distributed overlay routing state.³⁷

³²Anton V. Uzunov, “A Survey of Security Solutions for Distributed Publish/Subscribe Systems,” *Comput. Secur.* (GBR) 61, no. C (2016): 94–129, <https://doi.org/10.1016/j.cose.2016.04.008>, p 120.

³³Uzunov, “A Survey of Security Solutions for Distributed Publish/Subscribe Systems,” p 120.

³⁴Eugster et al., “The Many Faces of Publish/Subscribe,” p 10.

³⁵Uzunov, “A Survey of Security Solutions for Distributed Publish/Subscribe Systems,” p 120.

³⁶Uzunov, “A Survey of Security Solutions for Distributed Publish/Subscribe Systems,” p 119.

³⁷Uzunov, “A Survey of Security Solutions for Distributed Publish/Subscribe Systems,” p 111.

5. Scalability of blockchain systems

Prompt

Blockchain technology has become a much debated topic, often for very different reasons. Part of the popularity comes from the belief that blockchains can operate without the need for a trusted third party (such as a bank) while offering scalability. When diving deeper into the technicalities, there are serious problems. One specific problem that may eventually turn out to be too difficult to solve, is that the combination of full decentralization (i.e., no trusted third party), high transaction processing capabilities, scalability in the number of participants, and still attaining global consensus on the commitment of transactions is practically impossible to realize. One could argue that this impossibility renders blockchains practically useless.

Imagine the situation that the CEO of a software company is considering to tender for developing a general-purpose blockchain platform that can act as a middleware solution for a range of potential applications. Aware of the controversies around blockchains, they ask your expert team to give a well-founded advice on whether or not they should considering developing such a platform.

Recommendation

The CEO is strongly advised to cancel any development of a general-purpose blockchain platform, as the underlying architecture is bound by a fundamental distributed systems conflict known as the Blockchain Trilemma.³⁸ Achieving global transaction finality in a permission-less network requires all participating nodes to execute computationally expensive consensus mechanisms to resolve deterministic race conditions without a trusted intermediary.³⁹ When employing a race-based protocol, if the protocol's block-generation timeouts are configured too low, the overlay encounters severe network partitioning and availability issues;⁴⁰ conversely, long timeouts induce unacceptable transaction processing latencies that fail enterprise middleware standards. Because a general-purpose platform cannot predict the varying latency, scale, and consistency constraints of arbitrary target applications, building a universal substrate forces a structural compromise which inherently limits scalability.

This scalability bottleneck is driven by the strict voting and validation thresholds required to maintain a state machine replication ledger in an unauthenticated, peer-to-peer setting.⁴¹ To survive Byzantine faults where malicious nodes can actively spoof identities, inject malformed blocks, or poison routing entries,

³⁸Coinbase, *What Is the Blockchain Trilemma?*, <https://www.coinbase.com/learn/crypto-glossary/what-is-the-blockchain-trilemma>, 2026.

³⁹Juan Garay and Aggelos Kiayias, "SoK: A Consensus Taxonomy in the Blockchain Era," in *Topics in Cryptology – CT-RSA 2020*, ed. Stanislaw Jarecki (Springer International Publishing, 2020), https://doi.org/10.1007/978-3-030-40186-3_13.

⁴⁰Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems (4th Edition) Lecture Slides: Chapter 6 – Blockchain*, <https://www.distributed-systems.net/my-data/edu/06%20Blockchain.pdf>, 2023.

⁴¹Shehar Bano et al., *Consensus in the Age of Blockchains*, 2017, <https://arxiv.org/abs/1711.03936>, p 3.

classical consensus algorithms demand an overwhelming $3t + 1$ validator quorum to tolerate only t active failures.⁴² Forcing a wide-area network of un-vetted participants to execute these all-to-all messaging steps creates an $O(n^2)$ communication complexity overhead that quickly degrades transaction throughput as the node population expands.⁴³ These scaling issues will inevitably mean higher costs, unpredictable behavior, and customer dissatisfaction. Attempting to substitute this absolute ground-truth model with optimistic, multi-version branch structures - similar to Git repositories⁴⁴ - decouples the runtime race conditions but fundamentally destroys the ledger’s core purpose of maintaining a single, universally synchronized global state.

Instead of wasting critical development resources on a general-purpose infrastructure layer, the software company should pivot its strategy toward offering modular, domain-specific Byzantine fault-tolerant (BFT) sharding architectures.⁴⁵ By grouping verified client sets into closed, permissioned committees, the middleware can execute highly optimized, parallelized intra-committee atomic commits to achieve sub-second finality at bare-metal processing speeds. This sharded design safely bypasses the infinite data accumulation and power-intensive bottlenecks of open blockchains while preserving strong consistency and cryptographic verifiability for the specific enterprise applications that require it. Transitioning to a permissioned, multiple-committee ecosystem delivers immediate architectural utility to target industries without drowning the firm in the insurmountable technical debt of open-group identity tracking.

⁴²Steen and Tanenbaum, *Distributed Systems (4th Edition) Lecture Slides*.

⁴³Bano et al., *Consensus in the Age of Blockchains*, p 12.

⁴⁴Git, *Hash Function Transition*, <https://git-scm.com/docs/hash-function-transition>, 2026.

⁴⁵Bano et al., *Consensus in the Age of Blockchains*, p 12.

References

- Bano, Shehar, Alberto Sonnino, Mustafa Al-Bassam, et al. *Consensus in the Age of Blockchains*. 2017.
<https://arxiv.org/abs/1711.03936>.
- Coinbase. *What Is the Blockchain Trilemma?* <https://www.coinbase.com/learn/crypto-glossary/what-is-the-blockchain-trilemma>, 2026.
- Eugster, Patrick Th., Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. “The Many Faces of Publish/Subscribe.” *ACM Comput. Surv.* (New York, NY, USA) 35, no. 2 (2003): 114–31. <https://doi.org/10.1145/857076.857078>.
- Funda Engineering. *About Funda*. <https://blog.funda.nl/about/>, 2026.
- Garay, Juan, and Aggelos Kiayias. “SoK: A Consensus Taxonomy in the Blockchain Era.” In *Topics in Cryptology – CT-RSA 2020*, edited by Stanislaw Jarecki. Springer International Publishing, 2020.
https://doi.org/10.1007/978-3-030-40186-3_13.
- Gilbert, Seth, and Nancy Lynch. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.” *ACM SIGACT News* 33, no. 2 (2002): 51–59. <https://doi.org/10.1145/564585.564601>.
- Git. *Hash Function Transition*. <https://git-scm.com/docs/hash-function-transition>, 2026.
- Khan, Wazir Zada, Ejaz Ahmed, Saqib Hakak, Ibrar Yaqoob, and Arif Ahmed. “Edge Computing: A Survey.” *Future Generation Computer Systems* 97 (2019): 219–35. <https://doi.org/https://doi.org/10.1016/j.future.2019.02.050>.

Mouradian, Carla, Diala Naboulsi, Sami Yangui, Roch H. Glitho, Monique J. Morrow, and Paul A. Polakos.

“A Comprehensive Survey on Fog Computing: State-of-the-Art and Research Challenges.” *IEEE Communications Surveys & Tutorials* 20, no. 1 (2018): 416–64. <https://doi.org/10.1109/COMST.2017.2771153>.

Norberg, Arvid. *Introduction to BitTorrent*. Distributed systems C, TDBC85, Umeå University, 2006. <https://www.libtorrent.org/bittorrent.pdf>.

Randal, Allison. “The Ideal Versus the Real: Revisiting the History of Virtual Machines and Containers.” *ACM Computing Surveys* 53, no. 1 (2020): 1–31. <https://doi.org/10.1145/3365199>.

Rhea, Sean, Dennis Geels, Timothy Roscoe, and John Kubiawicz. “Handling Churn in a DHT.” *Proceedings of the USENIX Annual Technical Conference*, 2004. <https://www.cs.princeton.edu/courses/archive/fall109/cos518/papers/bamboo>.

Rhea, Sean, Brighten Godfrey, Brad Karp, et al. “OpenDHT: A Public DHT Service and Its Uses.” *Proceedings of the 2005 ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM ’05)* (New York, NY, USA), 2005, 73–84. <https://doi.org/10.1145/1080091.1080102>.

Sharma, Prateek, Lucas Chaufournier, Prashant Shenoy, and Y. C. Tay. “Containers and Virtual Machines at Scale: A Comparative Study.” *Proceedings of the 17th International Middleware Conference* (New York, NY, USA), Middleware ’16, 2016. <https://doi.org/10.1145/2988336.2988337>.

Steen, Maarten van, and Andrew S. Tanenbaum. *Distributed Systems (4th Edition) Lecture Slides: Chapter 6 – Blockchain*. <https://www.distributed-systems.net/my-data/edu/06%20Blockchain.pdf>, 2023.

Steen, Martinus Richardus van, and Andrew S. Tanenbaum. *Distributed Systems*. Amazon Fulfillment

Poland Sp. z o.o, 2017.

Urdaneta, Guido, Guillaume Pierre, and Maarten Van Steen. “A Survey of DHT Security Techniques.” *ACM Comput. Surv.* (New York, NY, USA) 43, no. 2 (2011). <https://doi.org/10.1145/1883612.1883615>.

Uzunov, Anton V. “A Survey of Security Solutions for Distributed Publish/Subscribe Systems.” *Comput. Secur.* (GBR) 61, no. C (2016): 94–129. <https://doi.org/10.1016/j.cose.2016.04.008>.